

Deep Learning and Its Application

Week 3: Practicals

1. Given

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
X, y = make_classification(n_samples=100, random_state=1)
```

Develop a classification model using Multi Perceptron class **MLPClassifier** in Scikit_Learn and display the probabilities $P(y|x)$, prediction results, and scores, loss function, activation function

Answer:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

#load data from Scikit-Learn
X, y = make_classification(n_samples=100, random_state=1)

#split the dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y,
random_state=1)

#construct a classification model
clf = MLPClassifier(random_state=1, max_iter=300)

# training
clf.fit(X_train, y_train)

#display the probabilities P(y|x)
print(clf.predict_proba(X_test[:1]))

#predict
print(clf.predict(X_test[:5, :]))

print(clf.score(X_test, y_test))
```

2. Given

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
X, y = make_regression(n_samples=200, random_state=1)
```

Develop a classification model using Multi Perceptron class **MLPRegression** in Scikit_Learn and display prediction results, and scores, loss function, activation function

Answer:

```

from sklearn.neural_network import MLPRegressor
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split

X, y = make_regression(n_samples=200, random_state=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)

#print(X)
#print(y)

regr = MLPRegressor(random_state=1, max_iter=500)
regr.fit(X_train, y_train)

print(regr.loss)
print(regr.activation)
print(regr.predict(X_test[:2]))
print(regr.score(X_test, y_test))

```

3. Given

```

from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test =
.
.
.

row2 = np.array([[5.1, 3.5, 1.4, 0.2]])

yhat = model.predict([row2])
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))

```

Use **Sequential** model to develop a multiclass classification model using Iris dataset, display

- a) Training accuracy
- b) Prediction results

Answer:

```
from numpy import argmax
from pandas import read_csv
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/iris.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# ensure all data are floating point values
X = X.astype('float32')
# encode strings to integer
y = LabelEncoder().fit_transform(y)
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(3, activation='softmax'))
# compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
loss, acc = model.evaluate(X_test, y_test, verbose=0)
print('Test Accuracy: %.3f' % acc)
# make a prediction
row1 = [5.1,3.5,1.4,0.2]

row2 = np.array([[5.1,3.5,1.4,0.2]])

yhat = model.predict([row2])
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

4. Given

```
from numpy import sqrt
from pandas import read_csv
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path =
'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# split into train and test datasets
X_train, X_test, y_train, y_test =
```

```

.
.
.

row2 =
np.array([[0.00632,18.00,2.310,0,0.5380,6.5750,65.20,4.0900,1,296.0,15.30,396.9
0,4.98]])
yhat = model.predict([row2])
print('Predicted: %.3f' % yhat)

```

Use **Sequential** model to develop a regression model using Boston Housing dataset, display

- c) Squared sum error
- d) Prediction results

Answer:

```

from numpy import sqrt
from pandas import read_csv
from sklearn.model_selection import train_test_split
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

# load the dataset
path = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
df = read_csv(path, header=None)
# split into input and output columns
X, y = df.values[:, :-1], df.values[:, -1]
# split into train and test datasets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
# determine the number of input features
n_features = X_train.shape[1]
# define model
model = Sequential()
model.add(Dense(10, activation='relu', kernel_initializer='he_normal',
input_shape=(n_features,)))
model.add(Dense(8, activation='relu', kernel_initializer='he_normal'))
model.add(Dense(1))
# compile the model
model.compile(optimizer='adam', loss='mse')
# fit the model
model.fit(X_train, y_train, epochs=150, batch_size=32, verbose=0)
# evaluate the model
error = model.evaluate(X_test, y_test, verbose=0)
print('MSE: %.3f, RMSE: %.3f' % (error, sqrt(error)))
# make a prediction
row1 = [0.00632,18.00,2.310,0,0.5380,6.5750,65.20,4.0900,1,296.0,15.30,396.90,4.98]
row2 =
np.array([[0.00632,18.00,2.310,0,0.5380,6.5750,65.20,4.0900,1,296.0,15.30,396.90,4.
98]])
yhat = model.predict([row2])
print('Predicted: %.3f' % yhat)

```